

GENERICS

```
// class
class Box<T> { }
// variable
Box<String> b = new Box<String>();
// method
public <T> void doStuff(T value) { }
// Full Example
class OrderedPair<T, U> {
    T first;
    U second;
    public OrderedPair(T t, U u) {
        this.first = t;
        this.second = u;
    }
    public T getFirst() { return first; }
    public U getSecond() { return second; }
    @Override
    public int hashCode() {
        return Objects.hash(first.hashCode(),
            second.hashCode());
    }
    @Override
    public boolean equals(Object obj) {
        if (obj == null ||
            obj.getClass() != getClass()) return false;
        @SuppressWarnings("unchecked")
        var other = (OrderedPair<T, U>)obj;
        return Objects.equals(first, other.first) &&
            Objects.equals(second, other.second);
    }
}

public class Employee implements Comparable<Employee>
{
    // ...
    @Override
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary);
    }
}
```

JAVA BEING STUPID

```
T obj = new T(); // error
obj instanceof T; // error
T heh = (T) "asdf"; // no typechecking
var s = Stack<int>(); // no primitive types
private static T x; // static no workey
Node<Number> s = new Node<Integer>; // error

class IThrowAnErrorBecauseWhyNot {
    void m(List<String> list) { }
    void m(List<Integer> list) { }
}
// error because of type erasure & identifiability
static <T> extends Comparable<T>> T m(T x, T y, T z) {
    return x > y ? z : x;
}

Integer n = m(1, 3.141, 0); // error
Number n = m(1, 3.141, 0); // ok
```

COMPARABLE

```
interface Comparable<T> {
    int compareTo(T o);
}

static <T> extends Comparable<T> doStuff(T a, T b) {
    // compareTo is possible with all types
    return a.compareTo("b") > 0 ? a : b;
}

static <T> extends Comparable<T>> T doStuff(T a,T b) {
    // compareTo is possible only with T
    return a.compareTo(b) > 0 ? a : b;
}
```

EXAMPLE

```
class Graphic {
    public void draw() { }
}

class Rectangle extends Graphic { }
class Circle extends Graphic { }
class GraphicStack<T> extends Graphic
    extends Stack<T> {
    public void drawAll() {
        for (T item : this) item.draw();
    }
}
```

BYTECODE

JVM Architecture:
– Operand Stack (LIFO op vals)
– Local variables (op results)
– Constant pool (runtime refs)

TYPE ERASURE

– Introduced because of "bAcKwArDs CoMpAtAbIlItY"
– No type information at runtime (Non-Reifiable Type)

```
class MyStack<T> {
    Entry<T> top;
    int push(T value) { }
}
// becomes
class MyStack {
    Entry top;
    int push(Object value) { }
}
/* with bounds */
class MyStack<T> extends Comparable<T>> {
    Entry<T> top;
    int push(T value) { }
}
// becomes
class MyStack {
    Entry top;
    int push(Comparable value) { }
}
/* what happens at/before runtime */
MyStack<String> s = new MyStack<String>();
s.push("Eh");
String x = s.pop();
// becomes
MyStack s = new MyStack();
s.push("Eh");
String x = (String) s.pop();
```

WILDCARDS

```
void printGs(List<Graphic> gs) { }
List<Graphic> gs1 = new ArrayList<>();
printGs(gs1); // ok
List<Circle> gs2 = new ArrayList<>();
printGs(gs2); // error
/* solution */
void printGs(List<? extends Graphic> gs) { }
```

GENERIC VARIANCE

	Type	Compatible Type-Args	R	W
Invariance	C<T>	T	✓	✓
Covariance	C<? extends T>	T and sub-types	✓	✓
Contravariance	C<? super T>	T and basis-types	✓	✓
Bivariance	C<?>	All	✗	✗

INVARIANCE

```
static <T> void move(Stack<T>, from, Stack<T> to) {
    while(!from.isEmpty()) to.push(from.pop());
}

var rectS = new Stack<Rectangle>();
var graphS = new Stack<Graphic>();
graphS.push(new Rectangle()); // ok
move(rectS, graphS); // error
Stack<Object> objS = graphS; // error
/* anotherone */
String[] strA = new String[10];
Object[] objA = strA; // ok
objA[0] = Integer.valueOf(2); // runtime err
/* anotherone */
ArrayList<String> sA = new ArrayList<>();
ArrayList<Object> oA = sA; // error
```

TODO:

```
// compiles?
List<Graphic> list = new ArrayList<Rectangle>();
// compiles?
Rectangle[] rectangleStack = new Rectangle[10];
Graphic[] graphicStack = new Graphic[10];
graphicStack = rectangleStack;
// compiles?
Stack<Rectangle> rectangleStack = new Stack<>();
Stack<Graphic> graphicStack = new Stack<>();
```

```
graphicStack = rectangleStack;
// compiles?
Stack<Rectangle> rectangleStack = new Stack<>();
Stack<Graphic> graphicStack = new Stack<>();
for (Rectangle r : rectangleStack) {
    graphicStack.push(r);
}
```

COVARIANCE

```
public class Stack<E> {
    public void pushAll(Iterable<? extends E> src) { }
}

Stack<? extends Graphic> graphS;
graphS = new Stack<Rectangle>(); // ok
graphS = new Stack<Circle>(); // ok
graphS = new Stack<Object>(); // error (duh)
/* read only */
graphS.push(new Graphic()); // error
graphS.push(new Rectangle()); // error
graphS.push(new Triangle()); // error
```

CONTRAVARIANCE

```
static <T> void addToC(List<? super T> list, T e) {
    list.add(e);
}

addToC(new ArrayList<Number>(), 3); // ok
addToC(new ArrayList<Object>(), 3); // ok
/* shenanigans */
Stack<? super Graphic> s = new Stack<Object>();
s.add(new Graphic()); // ok
s.add(new Rectangle()); // ok
Graphic g = s.pop(); // error
Object g = s.pop(); // ok
```

BIVARIANCE

– If concrete type doesn't matter

```
static void printList(List<?> list) {
    for (Object elem : list)
        System.out.print(elem + " ");
}

/* more shenanigans */
static void appendNewObject(List<?> list) {
    list.add(new Object()); // error
}
```

ANOTHERONE

```
public class VarianceExamples {
    public static double sum(
        Collection<? extends Number> numbers) {
        double sum = 0;
        for (Number num : numbers)
            sum += num.doubleValue();
        return sum;
    }

    public static void addNumbers(List<? super Integer>
        list, Collection<? extends Integer> source) {
        list.addAll(source);
    }

    public static <T> extends Comparable<T>> T
        findMax(Collection<T> coll) {
        return coll.stream()
            .max(Comparator.naturalOrder())
            .orElse(null);
    }

    public static <T> List<T> filterByType(
        Collection<?> source, Class<T> clazz) {
        List<T> result = new ArrayList<>();
        for (Object obj : source)
            if (clazz.isInstance(obj))
                result.add(clazz.cast(obj));
        return result;
    }
}
```

PRODUCER / CONSUMER

from **produces** entries, to **consumes** entries

```
<T> void move(Stack<? extends T> from,
    Stack<? super T> to) {
    while (!from.isEmpty())
        to.push(from.pop());
}
```

MISC

<T extends Comparable & Collection> // multiple type bounds

```
Set<Integer> set = new Set<>();
Integer[] c = set.toArray(new Integer[0]);
```

RECORDS

```
public record Person (String name, String address) {}
```

GENERICS STREAM TOMFOOLERY

```
List<? extends Media> mediaList;
public <S extends T> List<S>
    filterBySubType(Class<S> subtype) {
    return mediaList.stream()
        .filter(subtype::isInstance)
        .map(subtype::cast)
        .collect(Collectors.toList());
}
```

ANNOTATIONS

– Metadata
– Macros for the poor people forced to work corporate
– @Override, @Test, @Json...

DEFINING

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface Author {
    String name();
    String date() default "N/A";
}

@Author(name = "UwU", date = "idon't validate input")
public class Wee00o { }
```

TODO:

Examples Woche03

REFLECTION

– Information of Class (private and public): Methods, Attributes, Parent class, implemented interfaces
– Calling methods possible
– Usages: Debugger, Serialization, Frameworks, Remote Procedure Call, ORM

```
// all of these `throws SecurityException`
public Field[] getDeclaredFields()
public Method[] getDeclaredMethods()
public Constructor<?>[] getDeclaredConstructors()

System.out.println(dump(new Student("a", "b"), 0));
// {
//     firstName = a
//     lastName = b
// }
Class c = "s".getClass(); // java.lang.String
```

METHOD

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Init { }
// ...
public class User {
    private String needsInit;
    @Init
    private void initObj() {
        this.needsInit = "wowie";
    }
}
```

CLASS

TODO:

```
getDeclaredConstructor().newInstance()
```

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface JsonSerializable { }
// ...
@JsonSerializable
public class User { }
```

ATTRIBUTE

```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD})
public @interface JsonElement {
    public String key() default "";
}
// ...
public class User {
    @JsonElement
    private String firstName;
}
```

VALIDATION

```
@Min(value = 18, message = "Age must be ≥ 18")
@Max(value = 99, message = "Age must be ≤ 99")
private int age;
@NotNull(message = "Name cannot be null")
private String name;
```

EXAMPLE

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface WebController { }
// ...
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Get {
    String path();
    String contentType() default "application/json";
}
// ...
@WebController
public class Controller {
    private User user = new User("frank", "meier");
    @Get(path = "/user")
    public String user() {
        return new ObjectToJsonConverter()
            .convertToJson(user);
    }
}
// ...
public class WebFramework {
    public void addController(Class<?> controllerClass)
        throws Exception {
        if (!controllerClass
            .isAnnotationPresent(WebController.class))
            throw new Exception("Is not a WebController");

        for (Method method : controllerClass
            .getDeclaredMethods())
            if (method.isAnnotationPresent(Get.class)) {
                method.setAccessible(true); // best practice or
                sth, idk, i write code in actually good languages
                var annot = method.getAnnotation(Get.class);
                routeHandlers.put(
                    annot.path(),
                    new WebHandler<>(method,
                        controllerClass
                            .getDeclaredConstructor()
                                .newInstance(),
                                annot.contentType()));
            }
}
```

ALGORITHMS

Brute-force: slow, stupid, go shame yourself
Greedy: Take best option each step (optimize locally). Fast.
Not necessarily global optimum
Backtracking: Trial and error. Best overall option. Higher compute costs
Divide and conquer: Binary search
Dynamic programming: Recursion optimization? Tabulation. Iterative reuse of previous results

BACKTRACKING

TODO:
slides 87

TODO:
example

KNIGHTSTOUR

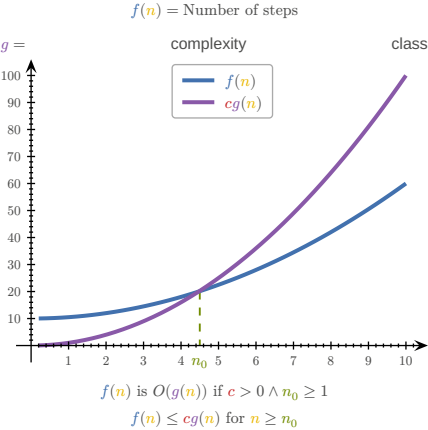
TODO:
diagram

```
boolean tour(int[][] visited, int x,int y, int pos) {
    visited[x][y] = pos;
    if (pos >= N * N) return true;
    for (int k = 0; k < 8; k++) {
        int newX = x + row[k];
        int newY = y + col[k];
        if (isValid(newX, newY)
            && visited[newX][newY] == 0
            && tour(visited, newX, newY, pos + 1))
            return true;
    }
    visited[x][y] = 0;
    return false;
}
```

BIG O NOTATION

TODO:
tricky examples

- Upper bound of f
 - Worst case scenario
 - Atomic operations = constant time
 - Runtime measured as sum of primitive operations
- Notation
- | | | | |
|-----|------------------|-----|-----------------|
| f | Algorithm | n | Size of Problem |
| g | Complexity Class | c | > 0 |
- $n_0 \geq 1$



RULES

If $f(n)$ is polynomial of degree d , then $f(n) \in O(n^d)$

- ignore lower powers
- ignore constants

Use most optimal function (lowest power)

- $2n$ is $O(n)$ better than $2n$ is $O(n^2)$

Simplify as far as possible

- $3n + 5$ is $O(n)$ instead of $3n + 5$ is $O(3n)$

COMPLEXITY CLASSES

Name	Class	Example
Constant	1	Array read
Logarithmic	$\log n$	Binary search
Linear	n	Linear search
Log-linear	$n \log n$	Merge+Quick sort
Quadratic	n^2	Bubble+Insertion sort
Cubic	n^3	Solve equation
Polynomial	n^k	Various algorithms
Exponential	2^n	Calculate all subsets
Factorial	$n!$	Calculate all permutations

MAFS

$$1 + 2 + \dots + n = \sum_{k=1}^n k = \frac{n(n+1)}{2}$$
$$1 + q + q^2 + \dots + q^n = \sum_{k=0}^n q^k = \frac{1 - q^{n+1}}{1 - q}$$

$q=2, n=\log_2(n)$
 $2n - 1$

SORTING ALGORITHMS

TODO:
Sorting diagrams

BOGOSORT

The best sorting algorithm

STALINSORT

The most efficient sorting algorithm

SELECTION SORT

Search for smallest elem in unsorted part and move in front

Less mutations in array

Same performance even if list almost sorted

```
public static void selectionSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int minimum = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minimum])
                minimum = j;
        swap(a, i, minimum);
    }
}
```

$(n - 1) + (n - 2) + \dots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \sim O(n^2)$

INSERTION SORT

Take elem and put into correct place in sorted part

Good for already partially sorted arrays

```
public static void insertionSort(int[] a) {
    int n = a.length;
    for (int i = 1; i < n; i++)
        for (int j = i; j > 0 && a[j] < a[j - 1]; j--)
            swap(a, j, j - 1);
}
```

$$1 + \dots + (n - 1) + n = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} \sim O(n^2)$$

BUBBLE SORT

Compare neighboring elems and swap if needed

```
public static void bubbleSort(int[] a) {
    for (n = a.length; n > 1; n--)
        for (i = 0; i < n - 1; i++)
            if (a[i] > a[i + 1])
                swap(a, i, i + 1);
}
```

$$(n - 1) + (n - 2) + \dots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \sim O(n^2)$$

COUNTING SORT

```
public static void countingSort(int[] a) {
    int k = Arrays.stream(arr).max().getAsInt();
    int[] c = new int[k + 1];
    for (int i = 0; i < a.length; i++) c[a[i]]++;
    int pos = 0;
    for (int i = 0; i < k; i++)
        for (int j = 0; j < c[i]; j++) {
            a[pos] = i;
            pos++;
        }
}
```

$$n + n + k + n = 3n + k \sim O(n + k)$$

SHELL SORT

Sort pairs of far away elems

Sort the partially sorted array through insertion sort

```
public static void shellSort(int[] a) {
    int n = a.length;
    int h = 1;
    while (h < n/3) h = 3 * h + 1;
    while (h >= 1) {
        for (int i = h; i < n; i++)
            for (int j = i; j >= h && a[j] < a[j - h]; j = j - h)
                swap(a, j, j - h);
        h /= 3;
    }
}
```

BINARY SEARCH

TODO:
iterative vs recursive (Woche04 Aufgabe3)

```
public static <T extends Comparable<T>> boolean bs(
    List<T> data, T target, int low, int high) {
    if (low > high) return false;
    int pivot = low + ((high - low) / 2);
    if (target.equals(data.get(pivot)))
        return true;
    else if (target.compareTo(data.get(pivot)) < 0)
        return searchBinary(data, target, low, pivot - 1);
    else
        return searchBinary(data, target, pivot + 1, high);
}
```

$O(\log n)$

RECURSION

When a function calls itself. FP <3

LINEAR RECURSION

Recursive call starts **at most one** further recursive call

RECURSIVE → ITERATIVE

```
static int[] reverseArrRec(int[] a, int i, int j) {
    if (i < j) {
        int temp = a[j];
        a[j] = a[i];
        a[i] = temp;
        reverseArray(a, i + 1, j - 1);
    }
    return a;
}
static int[] reverseArrIter(int[] a, int i, int j) {
    while (i < j) {
        int temp = a[j];
        a[j] = a[i];
        a[i] = temp;
        i = i + 1;
        j = j - 1;
    }
    return a;
}
```

TAIL RECURSION

Linear recursion where the recursive call is **last**

```
static int recsum(int x) {
    if (x == 0) return 0;
    else return x + recsum(x - 1);
    // no tail recursion because "+" is evaluated last
}
static int tailrecsum(int x, int total) {
    if (x == 0) return total;
    else return tailrecsum(x - 1, total + x);
}
```

BINARY RECURSION

All non-terminal calls have **two** recursive calls

```
static int binsum(int low, int high) {
    if (low > high)
        return 0;
    else if (low == high)
        return low;
    else {
        int mid = (low + high) / 2;
        return binsum(low, mid) + binsum(mid + 1, high);
    }
}
```

DATA STRUCTURES

ADT: Abstract Data Type (e.g. Interface). "What", not "How"

Data structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Doubly Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Binary Tree	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary Search Tree	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Hash Table	$O(n)$	$O(n)$	$O(n)$	$O(n)$

ARRAY

$\text{int}[] \text{aAaHrrray} = \{69, 420, 1337\};$

LINKED LIST

```
class LinkedList<T> {
    Node head;
    static class Node {
        T data;
        Node next;
        Node(T d) { data = d; }
    }
    public T insert(T data) {
        Node n = new Node(data);
        if (list.head == null)
            list.head = n;
        else {
            Node last = list.head;
            while (last.next != null)
                last = last.next;
            last.next = n;
        }
        return data;
    }
}
```

DOUBLY LINKED LIST

```
class DoublyLinkedList<T> {
    Node head;
    static class Node {
        T data;
        Node next, prev;
        Node(T d) { data = d; }
    }
    public T insert(T data) {
        Node n = new Node(data);
        if (list.head == null)
            list.head = n;
        else {
            Node last = list.head;
            while (last.next != null)
                last = last.next;
            n.prev = last;
            last.next = n;
        }
        return data;
    }
}
```

00P2 | FS26

Page 2

STACK

LIFO

1234

↕

```
class Stack<T> {
    private int maxSize;
    private T[] arr;
    private int top;
    public Stack(int size) {
        maxSize = size;
        arr = (T[]) new Object[maxSize];
        top = -1;
    }
    public void push(T value) {
        if (top == -1)
            throw new StackOverflowError();
        arr[++top] = value;
    }
    public T pop() {
        if (top == -1)
            throw new StackUnderflowError();
        return arr[top--];
    }
}
```

TREE

1

↙ ↘

2 3

4

↙ ↘

2 5

```
public class Tree<T> {
    private Node<T> root;
    public Tree(T rootData) {
        root = new Node<T>();
        root.data = rootData;
    }
    public static class Node<T> {
        private T data;
        private Node<T> left, right;
    }
}
```

QUEUE

FIFO

← 1234 →

```
class Queue<T> {
    private int maxSize;
    private T[] arr;
    private int front, rear;
    public Queue(int size) {
        maxSize = size;
        arr = (T[]) new Object[maxSize];
        front = rear = -1;
    }
    public T front() {
        if (front == -1)
            throw new StackUnderflowError();
        return arr[front];
    }
    public T rear() {
        if (rear == -1)
            throw new StackUnderflowError();
        return arr[rear];
    }
    public void enqueue(T value) {
        if (this.empty()) {
            front = 0;
            rear = 0;
            arr[front] = value;
        } else {
            front++;
            if (front < maxSize) arr[front] = value;
            else // rebuild array
        }
    }
    public T dequeue() {
        if (this.empty())
            throw new StackUnderflowError();
        T ret = arr[rear];
        if (front == rear) front = rear = -1;
        else rear++;
        return ret;
    }
    boolean empty() {
        return front == -1 && rear == -1;
    }
}
```

RING BUFFER

(front + elems) % cap

561234

```
class RingBuffer {
    private int[] buffer;
    private int size;
    private int start;
    private int end;
    private boolean isFull;

    public RingBuffer(int size) {
        if (size <= 0)
            throw new IllegalArgumentException("size > 0");
        this.size = size;
        this.buffer = new int[size];
        this.start = 0;
        this.end = 0;
        this.isFull = false;
    }

    public void append(int item) {
        buffer[end] = item;
        end = (end + 1) % size;
        if (isFull)
            start = (start + 1) % size;
        if (end == start)
            isFull = true;
    }

    public List<Integer> getData() {
        List<Integer> items = new ArrayList<>();
        if (!isFull) {
            for (int i = 0; i < end; i++)
                items.add(buffer[i]);
        } else {
            for (int i = start; i < size; i++)
                items.add(buffer[i]);
            for (int i = 0; i < end; i++)
                items.add(buffer[i]);
        }
        return items;
    }
}
```

PRIORITY QUEUE

– Saves priority with each element

– Returns elements with lowest/highest prio first

Method	Unsorted List	Sorted List
insert	$O(1)$	$O(n)$
min	$O(n)$	$O(1)$
removeMin	$O(n)$	$O(1)$

TODO:

impl

```
public interface Entry<K,V> {
    K getKey();
    V getValue();
}

public interface PriorityQueue<K,V> {
    int size();
    boolean isEmpty();
    Entry<K,V> insert(K key, V value);
    Entry<K,V> min();
    Entry<K,V> removeMin();
}
```

HEAP

– Binary tree

– Layers 0, ..., $h - 1$ fully populated, layer h filled from left

– Operations always lowest RHS so that tree stays consistent

Min-Heap:

Keys of nodes are **smaller** or equal than children's

Max-Heap:

Keys of nodes are **larger** or equal than children's

1

↙ ↘

3 4

5

↙ ↘

9 8

1

↙ ↘

3 4

5

↙ ↘

9 8

ENQUEUE

1

↙ ↘

3 4

5

↙ ↘

9 8

1

↙ ↘

3 4

5

↙ ↘

9 8

DEQUEUE

1

↙ ↘

3 4

5

↙ ↘

9 8

1

↙ ↘

3 4

5

↙ ↘

9 8

HEAP SORT

Dequeue elems from head into list, resorting tree on each iter

```
public class HeapSort {
    private static <T> int getLargest(T[] array, int n,
        int parent, Comparator<T> comparator) {
        int left = 2 * parent + 1;
        int right = 2 * parent + 2;
        int largest = parent;
        if (left < n && comparator.compare(array[left],
            array[largest]) < 0)
            largest = left;
        if (right < n && comparator.compare(array[right],
            array[largest]) < 0)
            largest = right;
        return largest;
    }

    static <T> void heapify(T[] arr,
        Comparator<T> comparator) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, i, comparator);
    }

    public static <T> void heapify(T[] array,
        int parent, Comparator<T> comparator) {
        int largest = getLargest(array, array.length,
            parent, comparator);
        if (largest == parent) return;
        T temp = array[parent];
        array[parent] = array[largest];
        array[largest] = temp;
        heapify(array, largest, comparator);
    }
}
```

```
static <T> void percolate(T[] array, int parent,
    int n, Comparator<T> comparator) {
    int largest = getLargest(array, n,
        parent, comparator);
    if (largest == parent) return;
    T temp = array[parent];
    array[parent] = array[largest];
    array[largest] = temp;
    percolate(array, largest, n, comparator);
}
```

```
static <T> void sort(T[] arr,
    Comparator<T> comparator) {
    int n = arr.length;
    heapify(arr, comparator.reversed());
    while (n > 1) {
        T temp = arr[0];
        arr[0] = arr[n - 1];
        arr[n - 1] = temp;
        n = n - 1;
        percolate(arr, 0, n, comparator.reversed());
    }
}
```

BINARY SEARCH TREE (BST)

– All subnodes of left subtree are **smaller** than root

– All subnodes of right subtree are **larger** than root

TRAVERSING

Preorder (W-L-R)

Postorder (L-R-W)

Inorder (L-W-R)

Breadth-First/Level-Order

MORE DATA STRUCTURES

Set:

Elements of same type, no duplicates, without order

Multiset:

Set with duplicates

Map:

KV-pairs with unique keys (put, get, remove $O(n)$)

Multimap:

Key can have multiple values

HASHING

Hash-function h maps e onto $[0, n - 1]$. $h(e)$ = position of e

hash code

$\xrightarrow{\text{compression function}}$

$\xrightarrow{\text{Index}}$

e

$\xrightarrow{\text{compression function}}$

$\xrightarrow{\text{Index}}$

Properties of good hash functions:

– Equal distribution

– Fast computation

– Constant time complexity

– Deterministic

– Incorporates as much information from key as possible

HASH FUNCTIONS

– Modulo primes

– Memory address (Object.hashCode() default)

– Byte/Integer cast ByteBuffer.wrap(b).getInt()

– Component sum (eg. sum string char codepoints)

– Polynomial accumulation $a_0 + a_1z + a_2z^2 + \dots + a_{n-1}z^{n-1}$

– $z = 33$ for strings

– Horner-Schema (identical, but easier to compute):

– $a_0 + z \cdot (a_1 + z \cdot (a_2 + \dots + z \cdot a_{n-1}))$

EQUALITY

It is generally necessary to override hashCode whenever equals is overridden

$x.equals(y) \Rightarrow x.hashCode() = y.hashCode()$

HASHING OBJECTS

– Horner schema with $z = 31$ and start = 17

@Override

public int hashCode() {

int result = 17;

for (Object field : fields) result = 31 * result

+ (field == null ? 0 : field.hashCode());

return result

}

ANOMALIES

– Empty spaces

– Collisions

– Overflow

MANAGING HASH COLLISIONS

CLOSED ADDRESSING/SEPARATE CHAINING

Each bucket is a list.

Search: $O(\text{entries/buckets})$

– Larger storage overhead

– Acceptable performance under high load

OPEN ADDRESSING

e "overflows" into next empty bucket

→ Must probe 2 times in example

TODO:

Problem: Primary Clustering

Linear probing:

$s(k, i) = h(k) + i$

Linear probing backwards:

$s(k, i) = h(k) - i$

Quadratic probing:

$s(k, i) = h(k) + i^2$

TODO:

diagram, slides 10,11

TODO:

deletion (slides 16-21)

– Only mark as "deleted"

– Move next best candidate to deleted position

TODO:

access (slides 22-27)

Probability for finding free spot:

n = entries

N = buckets

$a = \frac{n}{N}$ = probability: occupied

p = probing steps

$P(X = p) = a^{p-1}(1 - a)$

$E(X) = \sum_{x=1}^{\infty} p \cdot P(X = p) = \frac{1}{1 - a} = N \cdot r$ accesses

– Resize/Rehash compute intensive

– No storage overhead

– Bad performance under high load

CUCKOO-HASHING

$h_1(x) = x \bmod 5$

$h_2(x) = x/5 \bmod 5$

Insert:

If $h_1(x) = \text{null} \Rightarrow T_1$, else push y into T_2

If no cycle $\Rightarrow O(1)$, if rehash is needed $\Rightarrow O(n)$

(MAX_CYCLE)

Search:

$O(1) \Rightarrow$ only has to check 2 positions

Rehashing:

$x \bmod n \Rightarrow x \bmod (n + m)$

TODO:

EXTENDIBLE HASHING

TODO:

– Resize/Rehash easier

– Larger storage overhead

– Good performance under high load

// getting index

int idx = key.hashCode() & ((1 << globalDepth) - 1);

// when to rehash

if (bucket.getLocalDepth() == globalDepth)

growHashTable();

else

splitBlock();

// splitting block

int highBit = 1 << localDepth;

for (Entry e : page.entries) {

int h = e.key.hashCode();

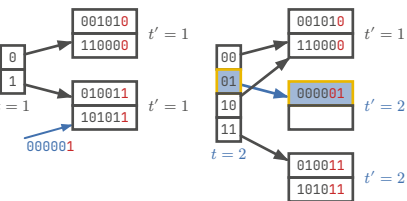
Page newPage = (h & highBit) == 0 ? p1 : p0;

newPage.put(e.key, e.value);

}

OOP2 | FS26

Page 3



DESIGN PATTERNS

- Creational: Abstraction and instantiation (e.g., Factory, Singleton)
- Structural: Composition of classes and objects into larger structures (e.g., Adapter, Facade)
- Behavioral: Algorithms and distribution of responsibility among objects (e.g., Iterator, Visitor)

ITERATOR

```
interface Iterable<T> {
    Iterator<T> iterator();
}

public interface Iterator<E> {
    boolean hasNext();
    E next() throws NoSuchElementException;
    void remove() throws UnsupportedOperationException,
        IllegalStateException;
}

for (String s : stringList) { }           // =
for (Iterator<String> i = stringList.iterator();
    i.hasNext(); ) String s = i.next();    // =
Iterator<String> iter = stringList.iterator();
while (i.hasNext()) String s = i.next();
// Tree → Depth-first Iterator
//      → Breadth-first Iterator
```

VISITOR

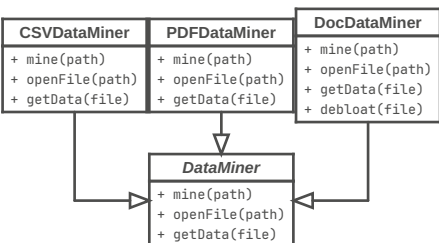
- separate algorithms and data
- keep algorithms in one place and not distributed over objs

```
public interface TagVisitor {
    void visit(HtmlTag html);
    void visit(HeadTag head);
    void visit(BodyTag body);
}

public class HtmlTag implements Tag {
    @Override
    public void accept(TagVisitor visitor) {
        visitor.visit(this);
        headTag.accept(visitor);
        bodyTag.accept(visitor);
        visitor.leave(this);
    }
}
```

TEMPLATE METHOD

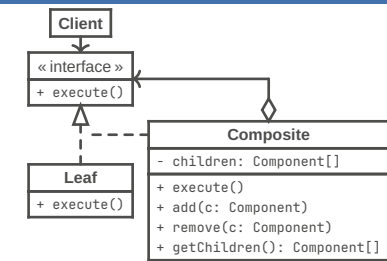
Break down an algorithm into a series of steps, turn these steps into methods, and put a series of calls to these methods inside a single template method.



EULER TOUR TRAVERSING

TODO:

COMPOSITE



```

interface Component {
    void printInfo();
}

class Item implements Component {
    private String name;
    public Item(String name) {
        this.name = name;
    }
    @Override
    public void printInfo() {
        System.out.println("Item: " + name);
    }
}

class Box implements Component {
    private List<Component> contents =
        new ArrayList<>();
    public void add(Component component) {
        contents.add(component);
    }
    public void remove(Component component) {
        contents.remove(component);
    }
    @Override
    public void printInfo() {
        for (Component c : contents)
            c.printInfo(); // Delegate to children
    }
}

```

TODO:

- Week 7 (knights, $O(n)$ (slides 84))
- Week 11
- Week 12
- Week 13 (euler traversing)

TODO:

Die Aussagekraft der O-Notation ist für kleine n unter Umständen begrenzt. Auch wenn zwei Algorithmen eine identische Laufzeitkomplexität (O-Notation) haben, kann sich ihre tatsächliche Laufzeit für kleine n unterscheiden. Warum ist das der Fall? Gib ein konkretes Beispiel. Betrachte zum Beispiel die Algorithmen Insertion Sort und Selection Sort oder lineare und binäre Suche. (4P)